

Trabajo Práctico Integrador

PROGRAMACIÓN 3

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

Alumno:

Lautaro Buchek

Fecha de Entrega:

02/07/2026

FoodStore

Link a video explicativo:

https://youtu.be/V1_uDeeolkY

Link a repositorio:

https://github.com/IBuKi19/Trabajo_Practico_Integrador-Programacion3

ÍNDICE

Marco Teorico.....	2
Decisiones Técnicas y Arquitectura.....	3-5
• Arquitectura Frontend.....	3-4
• Arquitectura Backend.....	4-6
Dificultades y soluciones.....	6-7
Bibliografía y Recursos.....	7

MARCO TEÓRICO

Tecnologías principales: TypeScript, Vite, HTML, CSS, Java, JPA, Hibernate, H2, Gradle.

Modalidad: Frontend Web + Backend de consola con persistencia JPA/Hibernate

Para el desarrollo frontend se utilizó HTML5, CSS3, TypeScript y Vite. HTML y CSS, que permitieron construir la estructura visual y los estilos de las pantallas del sistema, mientras que TypeScript aportó tipado estático sobre JavaScript, facilitando la detección temprana de errores y una mejor organización del código mediante interfaces y tipos. Vite se empleó como herramienta de desarrollo frontend por su rapidez, soporte para módulos modernos y facilidad para trabajar con proyectos TypeScript.

Para la gestión temporal de datos y por su finalidad académica, se utilizó localStorage, permitiendo almacenar la sesión del usuario, el carrito de compras y los pedidos generados desde el checkout. Esta API del navegador permite conservar datos entre sesiones dentro del mismo origen, lo cual fue útil para simular persistencia en la etapa frontend.

En el backend se utilizó Java junto con JPA, Hibernate y H2 Database. Java permitió modelar el dominio aplicando programación orientada a objetos, herencia, interfaces, enumerados y colecciones. JPA se utilizó como especificación para el mapeo objeto-relacional, permitiendo representar entidades como Categoría, Producto, Usuario, Pedido y DetallePedido.

Hibernate fue utilizado como implementación de JPA, encargándose de gestionar la persistencia de las entidades, las relaciones entre clases, las consultas JPQL y el manejo de transacciones. Esto permitió trabajar con operaciones CRUD, bajas lógicas y relaciones como Producto-Categoría y Pedido-DetallePedido.

La base de datos elegida fue H2, por ser aquella provista en los videos de la cátedra. Esta permitió persistir la información del backend sin necesidad de instalar un motor de base de datos externo, facilitando las pruebas del sistema desde consola.

Finalmente, aunque la consigna de backend se resolvió mediante una aplicación de consola con JPA/Hibernate, se contempla que en una futura etapa el frontend pueda reemplazar los fetch a archivos JSON por llamadas a una API REST desarrollada, por ejemplo, con Spring Boot 3.x. Spring Boot facilita la creación de aplicaciones Java modernas y se integra naturalmente con JPA, Hibernate y bases de datos relacionales.

DECISIONES Y ARQUITECTURA

El proyecto consta de dos partes independientes. Por un lado, el frontend web, desarrollado con Vite, TypeScript, HTML y CSS. Por otro lado, la estructura backend de consola, desarrollada con Java, JPA, Hibernate y H2.

Esta separación responde a la consigna, que propone construir primero un frontend desacoplado que consume archivos JSON locales y, por otro lado, un backend persistente con menú de consola. De este modo, el frontend pudo verificarse sin depender de una API REST, mientras que el backend pudo centrarse en persistencia, relaciones JPA, transacciones y consultas JPQL.

Arquitectura Frontend

Organizado según vistas de la aplicación y responsabilidades

```
src/  
├── types/  
├── utils/  
└── pages/  
    ├── auth/  
    ├── store/  
    ├── client/  
    └── admin/
```

La carpeta *types* contiene las interfaces TypeScript del sistema. La carpeta *utils* centraliza lógica reutilizable como autenticación, carrito, pedidos, productos y navegación. La carpeta *pages* contiene la lógica específica de cada pantalla.

Esta estructura permitió separar:

- Lógica de autenticación.
- Lógica de carrito.
- Fetch de datos.
- Renderizado específico por página.
- Tipos compartidos.

Debido a funciones como *fetchProductos* y *fetchCategorias*, que son utilizadas en múltiples instancias de la aplicación para solicitar la información de los documentos JSON, se optó por definir las en archivos apartados dentro del package *utils* para que luego puedan ser reutilizadas fácilmente. Lo mismo ocurre para el fetch a pedidos y usuarios, que se ubican dentro del archivo *order.ts* ubicado también en el package *utils*.

Debido a que las operaciones de consumo de datos se concentran en estos archivos dentro de utils, una futura integración con una API sería muy sencillo de implementar. En este caso, solo habría que reemplazar las rutas `/data/*.json` por endpoints REST del backend, como `/api/productos`, `/api/categorias`, `/api/usuarios` o `/api/pedidos`.

Autenticación y roles

La autenticación del frontend se implementó comparando las credenciales ingresadas contra `usuarios.json`. Cuando el usuario inicia sesión correctamente, se guarda un objeto `authUser` en `localStorage`, sin incluir la contraseña.

Se definieron dos roles según consigna:

- ADMIN
- USUARIO

El rol determina la redirección posterior al login:

- ADMIN → Panel de administración.
- USUARIO → Catálogo de productos.

También se implementó un guard para proteger vistas privadas y evitar que usuarios no autenticados o sin permisos accedan a páginas restringidas.

Arquitectura Backend

```
src/main/java/  
├── com.tp.jpjpa/  
│   ├── model/  
│   │   ├── enums/  
│   ├── repository/  
│   ├── util/  
│   └── Main.java
```

Las entidades representan el modelo de dominio, mientras que los enums modelan valores fijos como rol, estado y forma de pago. Los repositorios concentran la lógica de persistencia y operaciones CRUD, para luego en la clase Main desplegar el menú de consola con los distintos submenús correspondientes a las entidades.

Repositorio Base

Se implementó un `BaseRepository<T>` genérico para evitar repetir operaciones CRUD en cada entidad. Este repositorio concentra operaciones comunes como:

- guardar
- buscarPorId

- listarActivos
- eliminarLogico

Luego, repositorios específicos como CategoriaRepository, ProductoRepository, UsuarioRepository y PedidoRepository extienden esta clase base.

Esta decisión permite reutilizar código, mejorar la mantenibilidad y centralizar el manejo de EntityManager, transacciones y cierres de recursos. La consigna exige precisamente un BaseRepository<T> con CRUD genérico, transacciones, Optional, JPQL para activos y cierre del EntityManager en finally.

MAIN

La clase main(), se distribuye a partir de un menú principal que llama a los métodos, que contienen los diferentes menús según entidad (categoría, producto, usuario, pedido).

- menuCategorias
- menuProductos
- menuUsuarios
- menuPedidos
- menuReportes

Debido a que muchas de las operaciones realizadas en los menús van a repetir porciones de código, se implementaron métodos adicionales para listar los activos según la entidad.

- mostrarProductos
- mostrarUsuarios
- mostrarPedidos
- mostrarCategorias

ALTA DE PEDIDO

El alta de pedido fue implementada como una operación transaccional. Primero se selecciona un usuario activo, una forma de pago y una lista temporal de productos con sus cantidades. Esa lista temporal solo guarda IDs y cantidades para evitar mezclar entidades de distintos EntityManager. Luego se abre un único EntityManager y se inicia una transacción. Dentro de esa transacción se recuperan el usuario y los productos, se validan disponibilidad y stock, se crean los detalles del pedido, se calculan subtotales y total, se

descuenta stock y se asocia el pedido al usuario. Por último, se realiza el commit, y en caso de cualquier error, se hace rollback completo.

Separación de lógica y transacción

- altaPedido
- guardarPedidoTransaccional

La lógica de interacción por consola y el bucle de selección de productos para ser añadidos al pedido, se encuentra en el método de alta. Mientras que el inicio de la transacción, persistencia de pedido y commit, reside en el método de guardarPedidoTransaccional. De igual manera, se encuentran implementados algunos métodos helper adicionales como:

- seleccionarFormaPago
- obtenerCantidadTemporal
- agregarOactualizarItemTemporal

DIFICULTADES Y SOLUCIONES

Diferencias entre estructura del JSON y tipos TypeScript

Una dificultad inicial fue que algunos archivos JSON utilizaban nombres de propiedades distintos a los tipos definidos originalmente. Por ejemplo, usuarios.json utilizaba mail y rol, mientras que parte del código usaba email y rol. Como solución, se unificó el modelo del frontend para trabajar con mail y rol, respetando la estructura provista por la consigna y evitando errores como acceder a propiedades inexistentes.

Registro en localStorage y login contra JSON

El registro de usuarios no debía modificar usuarios.json, sino crear usuarios localmente. Esto generó la necesidad de combinar datos provenientes del JSON con datos almacenados en localStorage. Como solución, se ajustó el flujo para que los usuarios registrados se agreguen a localStorage, y se revisó el login para poder contemplar usuarios base y usuarios locales.

Múltiples DetallePedido al implementar alta de pedido

Al realizar las pruebas de alta de pedido, añadiendo diferentes productos al detalle, el pedido se creaba correctamente pero únicamente guardando el detallePedido del primer producto agregado, sin tener en cuenta el resto que se añadieron después.

El origen del problema estaba en que la entidad Pedido utiliza una colección de tipo Set<DetallePedido> para almacenar sus detalles. Al trabajar con un HashSet, Java necesita poder distinguir correctamente un objeto de otro mediante los métodos equals() y hashCode(). Sin embargo, DetallePedido estaba configurado con Lombok usando @EqualsAndHashCode sin incluir un campo que permitiera diferenciar detalles nuevos antes de ser persistidos. Como los detalles todavía no tenían ID generado por la base de datos, el Set podía considerar que dos detalles distintos eran iguales y conservar solo uno. Como solución, se ajustó la configuración de Lombok en la clase DetallePedido, excluyendo relaciones problemáticas del toString() y definiendo explícitamente qué campo debía usarse para diferenciar instancias dentro del Set.

BIBLIOGRAFÍA Y RECURSOS

- Material Teórico provisto por la cátedra
- Material Videográfico provisto por la cátedra
- Inteligencia Artificial utilizada: Microsoft Copilot

-Java Persistence API y contextos de transacciones locales. Disponible en:

<https://www.ibm.com/docs/es/was-liberty/nd?topic=liberty-java-persistence-api-jpa>

-JPA: ¿Qué es y Cómo Funciona la Java Persistence API? . Disponible en:

<https://www.arquitecturajava.com/jpa-un-ejemplo-desde-cero/>

-Material videográfico(Curso) : Java Persistence API -Tutorial Paso a Paso (TodoCode).
Disponible en :

https://www.youtube.com/watch?v=ppDv4N5A31E&list=PLQxX2eiEaqbxUrYkcsLADy zpX8ewgn_bM